

19A – Kernel Services Implementation Guide

Sprint 0, part two. The ten layer-0 services behind the interfaces 17A scaffolded: scope and session, identity, permissions, audit, event backbone, internationalisation, documents and numbering, edge sync gateway, attachments, notifications delivery, configuration register and scheduling – as tables, classes, algorithms, tests and tickets.

Field	Value
Document	19A – Kernel Services Implementation Guide (implements 19 Kernel Services v0.2, doc 18 parts C–E, doc 32)
Version	v0.1 – for the platform pair; Sprint 0 weeks 1–3 overlapping 17A from day 4
Date	8 September 2026
Prerequisite	17A: repository, kernel package with interfaces and stubs, baseline migration, hello module. Every section below replaces a 17A stub and must keep the hello module green.
Audience	Platform pair, DevOps, till track lead (sections 9 and 10 define what the till talks to).
Restated content	Marked <i>restated from doc X §Y</i> . Tables are given as DDL fragments; the full files live in <code>backend/app/src/main/resources/db/migration/kernel/</code> .

What done looks like. The hello module's integration tests pass against real services: a scoped user cannot read another entity's row; a resent command returns the same result; one command produces exactly one audit row and one outbox row, and a registered consumer sees the event once even when two instances run; a simulated till enrolls, uploads a batch, gets an acknowledgement, resends it and gets DUPLICATE; a snapshot delta is produced from a change-log row; a missing Tamil string renders in English with the fallback marker; a job runs once across two instances.

0. Service map: interface, tables, classes, ticket

Service	kernel.api interface (17A)	Tables (schema kernel)	Main classes (kernel.internal)	Ticket
1 Scope and session	ScopeContext	– (GUC helpers from 17A)	ScopeFilter, ScopeConnectionCustomizer, GrantedEntities	K-01
2 Identity	IdentityProviderClient, DeviceCredentials	– (provider holds credentials)	JwtClaimsMapper, StepUpInterceptor, PinHasher, DeviceAuth	K-02
3 Permissions	PermissionResolver, Sod	(security schema is M1's; kernel reads)	PermissionCache, SodChecker, TillSnapshotSigner	K-03
4 Audit	AuditFacade	audit_event, audit_event_type	AuditFacadeImpl, CorrelationFilter, ExceptionQueueQuery, PartitionJob	K-04
5 Event backbone	EventPublisher, @EventConsumer	event_outbox, event_inbox, source_sequence	OutboxWriter, OutboxRelay, BrokerAdapter, InboxGuard, DeadLetter	K-05
6 Internationalisation	Messages, TextNormaliser, Formats	message_catalogue (optional; files by default)	IcuMessages, NfcConverter, IcuFormats, A4Renderer	K-06
7 Documents and numbering	DocumentService, NumberingService, DocumentLinks	document_type, numbering_series, document, document_line, document_link, document_state_history, document_attachment	IssuanceProtocol, SeriesCounter, GapCheckJob, ImmutabilityTrigger (SQL)	K-07
8 Edge sync gateway	(REST) SyncApi	device_sync_cursor, sync_quarantine, snapshot_change_log	EnrolmentService, BatchIngestor, SnapshotBuilder, HeartbeatService, RateLimiter	K-08
9 Attachments	Attachments	document_attachment (part of 7)	PresignService, AttachmentVerifier	K-09
10 Notifications delivery	NotificationDispatcher	notification_log	RuleEvaluator, Dispatcher, RetrySweep	K-10
11 Configuration register	ConfigRegistry	config_item, config_value	ConfigResolver, ConfigCache, SeedLoader	K-11
12 Scheduling	@ScheduledJob	scheduled_job, job_run	JobRunner, LockProvider, DayCloseTrigger	K-12
13 Business date and clock	Clock, BusinessDate	–	BusinessDateService, ClockOffset	K-13

1. Scope and session

Restated from doc 19 §1 and doc 18 §3.7. Every request carries a resolved principal and scope; the database session carries the same facts as custom settings that the row-level security policies read.

```
// kernel.api.ScopeContext
public record ScopeContext(UUID userId, UUID deviceId, UUID homeEntityId,
    List<Scope> scopes, PolicyClass policyClass,
    Set<UUID> grantedEntities, Instant mfaAt, String lang, UUID correlationId) {
    public UUID entityId() { return activeScope().entityId(); } // the scope chosen for this request
    public UUID locationId() { return activeScope().locationId(); } // null for entity-wide
}

// kernel.internal.ScopeFilter (web profile): token -> ScopeContext -> request attribute + MDC
// kernel.internal.ScopeConnectionCustomizer: at transaction start, on the same connection:
// SET LOCAL app.scope_entity_id = '<uuid>'; SET LOCAL app.scope_location_id = '<uuid or empty>';
// SET LOCAL app.scope_class = 'OWN' | 'PARTY' | 'FEDERATION_VIEW' | 'EXTERNAL_TIMEBOXED';
// SET LOCAL (not SET) because PgBouncer runs in transaction pooling: settings must die with the transaction.
```

Aspect	Design
Active scope	A user with several scopes (Federation view, MPCs administrator of many shops) chooses one per request through the X-Scope-Entity / X-Scope-Location headers; the filter verifies the choice is among the token's scopes; a missing header with exactly one scope defaults to it; otherwise 400 scope.required.
Worker and ingest profiles	No HTTP; the job runner and the batch ingestor construct a ScopeContext explicitly (system actor, entity from the job or the device) and call the same customizer. A transaction without a scope fails closed: the customizer sets app.scope_class = 'NONE' and RLS returns nothing.

Aspect	Design
External grants	<code>kernel.granted_entities()</code> reads <code>app.granted_entities</code> (a uuid array set by the customizer from the token's grant list); used only by the <code>EXTERNAL_TIMEBOXED</code> policies.
PARTY policies	Document tables carry seller and buyer columns; the PARTY policy for SELECT is <code>USING (kernel.scope_class() IN ('OWN', 'PARTY') AND (owner_entity_id = kernel.scope_entity() OR counterparty_entity_id = kernel.scope_entity()))</code> ; the masking view <code>trading.v_document_party</code> omits cost, margin and internal notes and is what PARTY reads go through (implemented by M4 against the template here).
Tests	The RLS matrix of 17A §12 extended: OWN, PARTY, FEDERATION_VIEW, EXTERNAL grants, NONE; a transaction that forgets the customizer sees zero rows.

2. Identity

Restated from doc 19 §2. Four principals, four authentication paths; the provider issues tokens, the kernel maps claims and enforces step-up and device auth.

Piece	Implementation
Resource server	Spring Security OAuth2 resource server with the provider's JWKS; <code>JwtClaimsMapper</code> turns claims <code>sub</code> , <code>dev</code> , <code>ent</code> , <code>scopes[]</code> , <code>roles[]</code> , <code>rv</code> , <code>cls</code> , <code>mfa_at</code> , <code>lang</code> into <code>ScopeContext</code> (<i>doc 19 §1 claim table</i>). Claims <code>scopes</code> and <code>roles</code> are put into the token by a provider mapper that reads M1's <code>user_role</code> through a small kernel endpoint at token issue time, so the provider stays generic.
Provider wrapper	<code>IdentityProviderClient</code> (<code>kernel.api</code>): <code>createUser</code> , <code>disableUser</code> , <code>setTemporaryPassword</code> , <code>resetTotp</code> , <code>revokeSessions</code> . Implementation <code>KeycloakAdminClient</code> uses a service account limited to the users of the realm; the kernel enforces the scope rule (caller's entity must equal the target user's home entity, or be its MPSC parent for shop users) before any call, so no Federation role can reach the raw admin API (<i>ADR-18</i>).
Step-up MFA	<code>@CommandHandler(permission=..., requiresMfa=true)</code> or <code>permission.requires_mfa</code> in the catalogue; <code>StepUpInterceptor</code> checks <code>mfaAt</code> against the freshness window from the configuration register (<code>security.mfa_freshness_hours</code> , per-action list) and returns 401 with problem type <code>step_up_required</code> and the provider's step-up URL; the web shell handles it (<i>17A §7</i>).
Device credentials	Client certificate at the edge (mTLS terminated at the proxy, subject forwarded in a header the app trusts only from the proxy) or OAuth2 client credentials per device; either yields a device token with <code>dev</code> and <code>cls = DEVICE</code> . <code>DeviceAuth</code> validates status <code>ACTIVE</code> against M1 through the permission cache and rejects <code>SUSPENDED</code> with 403 plus the signed revoke instruction (<i>doc 32 §9</i>).
Till PIN	<code>PinHasher</code> : Argon2id, memory 64 MB, iterations 3, parallelism 2, 16-byte salt, output 32 bytes; the hash is what the operator snapshot carries. Verification happens on the device; the kernel never sees PINs at login.
Password policy	Configured in the provider: minimum 12 characters, breached-password check, no periodic rotation, rate limits (<i>NIST 800-63B</i>); the kernel only relays resets.
Tests	Claims mapping fixtures; step-up window boundaries; scope rule of the wrapper (an MPSC admin cannot reset a Federation user); device token with suspended status.

3. Permissions

Restated from doc 19 §3. The catalogue and roles are M1's data (schema `security`, *doc 21*); the kernel resolves and enforces.

```
// kernel.api.PermissionResolver
public interface PermissionResolver {
    Resolved resolve(UUID userId, int catalogueVersion);          // union of role grants per scope, with limits
    boolean allows(ScopeContext ctx, String permission);          // scope-aware check used by the interceptor
    Optional<Limits> limits(ScopeContext ctx, String permission); // e.g. approval ceilings
}
// kernel.internal.PermissionCache: Caffeine, key (userId, rv), TTL 10 min, invalidated by role.changed.v1,
//   role.assigned/revoked.v1, user.deactivated.v1 fanned out over the broker (doc 14 1.3).
// kernel.internal.CommandInterceptor: for every @CommandHandler -> permission check -> MFA check ->
//   idempotency check -> handler -> record result. Order fixed; tested.
// kernel.api.Sod.assertDistinct(ctx, "inv.writeoff.request", "inv.writeoff.approve", requesterId)
//   -> ProblemException("sod.same_person") when INSTANCE pair and requesterId == ctx.userId()
// kernel.internal.TillSnapshotSigner: builds {operators:[{userId, displayName, pinHash, permissions[]}], rv}
//   filtered to offline_allowed, signed as a JWS with the kernel's signing key; verified on the till.
```

Aspect	Design
Guardrails at role authoring	Enforced in M1's handlers with kernel helpers: <code>Sod.roleModeConflict(role)</code> , <code>Grant.withinGrantor(ctx, permissions)</code> , <code>Grant.lastAdminGuard(entity)</code> .
Scope of a permission	The catalogue row's scope (<code>LOCATION</code> , <code>ENTITY</code> , <code>FEDERATION</code>) decides the finest grantable level; the resolver expands an <code>ENTITY</code> grant to all the entity's locations at check time (<i>doc 19 §3.1</i>).
Tests	Resolver property test: union over roles never yields a permission absent from every role; cache invalidation on each event; interceptor order; SoD <code>INSTANCE</code> and <code>ROLE</code> modes.

4. Audit

Restated from doc 18 part D and doc 19 §4.

```
-- V0002__audit.sql (kernel)
CREATE TABLE kernel.audit_event_type (
  event_type_code varchar(40) PRIMARY KEY, severity text NOT NULL CHECK (severity IN
    ('INFO', 'REVIEW', 'ALERT')),
  reviewer_role_template uuid, offline_capturable boolean NOT NULL DEFAULT false, description_en text NOT
    NULL);
CREATE TABLE kernel.audit_event (
  audit_id uuid NOT NULL, event_type_code varchar(40) NOT NULL REFERENCES kernel.audit_event_type,
  occurred_at timestampz NOT NULL, occurred_local timestamp, received_at timestampz NOT NULL DEFAULT now(),
  owner_entity_id uuid NOT NULL, location_id uuid, actor_user_id uuid, actor_system varchar(40),
  device_id uuid, till_position_id uuid, subject_table varchar(40) NOT NULL, subject_id uuid NOT NULL,
  document_id uuid, before_state jsonb, after_state jsonb, reason_code text, reason_text text,
  witness_user_id uuid, device_seq bigint, correlation_id uuid NOT NULL, prev_hash char(64), row_hash
    char(64),
  PRIMARY KEY (received_at, audit_id)
) PARTITION BY RANGE (received_at);
CREATE INDEX ON kernel.audit_event (owner_entity_id, received_at DESC);
CREATE INDEX ON kernel.audit_event (document_id) WHERE document_id IS NOT NULL;
CREATE INDEX ON kernel.audit_event (event_type_code, received_at DESC);
ALTER TABLE kernel.audit_event ENABLE ROW LEVEL SECURITY; ALTER TABLE kernel.audit_event FORCE ROW LEVEL
  SECURITY;
-- policies: same four classes as any operational table (17A 6.3)
GRANT INSERT ON kernel.audit_event TO app_rw; -- and nothing else (doc 18 D-I1); SELECT via kernel.v_audit
  for readers
REVOKE UPDATE, DELETE, TRUNCATE ON kernel.audit_event FROM app_rw;
```

Aspect	Design
Facade	AuditFacadeImpl.record(type, subject, before, after, scope, reason?, witness?) validates the type against the catalogue, requires reason when the type demands it, computes the minimal diff of changed columns, fills actor, device, correlation from ScopeContext, and inserts in the caller's transaction. No queue.
Correlation	CorrelationFilter creates or propagates X-Correlation-Id; the ingestor uses the till's correlation from the envelope; jobs create one per run.
Exception queue	ExceptionQueueQuery.list(scope, severityIn, since) reads REVIEW/ALERT rows not yet acknowledged; acknowledgement is a further audit event AUDIT_REVIEWED with subject_id = the reviewed row (doc 19 §4.3); the self-review guard uses Sod.
Partitions	PartitionJob (Section 12) creates monthly partitions three months ahead for audit_event, event_outbox and the document tables; detaches partitions older than the tiering policy to cold storage (doc 14 DR-6) – detach, never drop.
Hash chain (P2)	Columns reserved; AnchorJob skeleton present but disabled; when enabled, computes per-partition chain and writes the head to the write-once bucket.
Privilege test	An integration test attempts UPDATE, DELETE and TRUNCATE as app_rw and asserts failure (D-I1).

5. Event backbone

Restated from doc 19 §6, doc 18 part E, doc 17 ADR-30.

```
-- V0003__events.sql (kernel)
CREATE SEQUENCE kernel.central_source_seq; -- source_seq for source = 'central'
CREATE TABLE kernel.event_outbox (
  event_id uuid NOT NULL, event_type text NOT NULL, occurred_at timestampz NOT NULL, occurred_local
    timestamp,
  received_at timestampz NOT NULL DEFAULT now(), source text NOT NULL, source_seq bigint NOT NULL,
  owner_entity_id uuid NOT NULL, location_id uuid, aggregate_type text NOT NULL, aggregate_id uuid NOT NULL,
  correlation_id uuid NOT NULL, causation_id uuid, actor_user_id uuid, engine_version text,
  payload jsonb NOT NULL, published_at timestampz,
  PRIMARY KEY (received_at, event_id), UNIQUE (source, source_seq, received_at)
) PARTITION BY RANGE (received_at);
CREATE INDEX ON kernel.event_outbox (published_at) WHERE published_at IS NULL; -- relay scan
CREATE TABLE kernel.event_inbox (consumer text NOT NULL, event_id uuid NOT NULL, applied_at timestampz NOT
  NULL DEFAULT now(),
  outcome text NOT NULL CHECK (outcome IN ('APPLIED', 'SKIPPED', 'FAILED')), last_error text, PRIMARY KEY
    (consumer, event_id));
GRANT INSERT ON kernel.event_outbox TO app_rw; GRANT UPDATE (published_at) ON kernel.event_outbox TO
  app_relay;

// kernel.internal.OutboxWriter (implements EventPublisher): in the caller's transaction
```

```
// source='central', source_seq = nextval(central_source_seq), envelope from shared-engine Envelope, payload
JSON
// kernel.internal.OutboxRelay (@ScheduledJob "outbox-relay", continuous, lock per partition of source):
// loop: SELECT ... WHERE published_at IS NULL ORDER BY source, source_seq LIMIT 500 FOR UPDATE SKIP LOCKED
// publish each to exchange "domain" with routing key = event_type (topic exchange), persistent
// UPDATE published_at = now()
// at-least-once: a crash after publish before update republishes; consumers are idempotent.
// kernel.api.EventConsumer(types = {"grn.confirmed.v1"}, consumer = "m5.receipts")
// framework: RabbitMQ queue per consumer bound to its types; on message ->
// InboxGuard: INSERT INTO event_inbox ON CONFLICT DO NOTHING -> if not inserted: ack & skip
// handler in a transaction with a system ScopeContext scoped to the event's entity
// failure: nack with retry count; after 3 -> dead-letter queue "domain.dlq" + ALERT audit event
// Replay: kernel.internal.Replayer.replay(consumer, fromSeq) republishes from the archive to one consumer's
queue.
```

Aspect	Design
Broker adapter	BrokerAdapter interface with RabbitBrokerAdapter (quorum queues, publisher confirms) and a NatsBrokerAdapter stub, so doc 19 DR-2 is a swap.
Ordering	Per source only: one relay lock per source partition; consumer queues are single-active-consumer so a consumer processes one source in order (<i>doc 19 §6.2</i>).
Payload discipline	A test rejects payloads over 8 KB and payloads containing fields named like personal data (name, phone, nic) outside a whitelist.
Tests	Two instances, one relay lock; crash injection between publish and update yields exactly one application per consumer; dead-letter after three failures; replay rebuilds a projection.

6. Internationalisation

Restated from doc 19 §5.

Aspect	Design
Messages	IcuMessages loads i18n/{en, si, ta}.json at start (ICU4J MessageFormat); t(id, locale, args) returns Text(value, fallback: boolean); a missing id in the requested language falls back to English with fallback = true; a missing id everywhere throws in tests and logs in production.
Normalisation	NfcConverter as a JPA AttributeConverter applied to every text column through a package-level @Converter(autoApply) on a marker type I18nText; the till applies the same rule in Kotlin (Normalizer.normalize(s, NFC)).
Collation and search	Name columns declared COLLATE si_icu / ta_icu / en_icu in module migrations (helper macro in the migration conventions); trigram indexes USING gin (name_si gin_trgm_ops) etc.; SearchSupport.orderByDisplayLanguage(query, locale) picks the collation.
Formats	IcuFormats: dates dd/MM/yyyy, money Rs 1,234.00, Western Arabic digits forced by NumberingSystem.LATN for si-LK and ta-LK (doc 19 DR-6).
A4 rendering	A4Renderer: HTML templates (Thymeleaf) rendered to PDF by a headless Chromium worker (Playwright) with bundled Noto fonts – chosen over pure-Java renderers because Sinhala and Tamil shaping needs HarfBuzz-class shaping; DR-3 confirms the operational footprint. Output to object storage; report.rendered.v1.
Input-method register	A configuration item location.input_methods (Section 11) recorded at onboarding; no runtime behaviour.
Tests	The curated Sinhala/Tamil name list of doc 19 §5.4 with expected sort orders and equivalent spellings; golden PDF rendering of a trilingual invoice compared by text extraction; fallback marker propagation through a message with plurals.

7. Documents and numbering base

Restated from doc 18 part C. Tables as in doc 18 §5 (document_type, numbering_series, document, document_line, document_link, document_state_history, document_attachment), partitioned by month on received_at for the volume tables; the DDL is in V0004__documents.sql. The behaviour that matters is the issuance protocol and immutability.

```
-- immutability (doc 18 C-I1): a trigger raises on any UPDATE of an issued document except status
CREATE OR REPLACE FUNCTION kernel.document_immutable() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF OLD.issued_at IS NOT NULL AND (to_jsonb(NEW) - 'status') IS DISTINCT FROM (to_jsonb(OLD) - 'status') THEN
        RAISE EXCEPTION 'document.immutable' USING ERRCODE = 'check_violation';
    END IF; RETURN NEW;
END $$;
CREATE TRIGGER trg_document_immutable BEFORE UPDATE ON kernel.document FOR EACH ROW EXECUTE FUNCTION
    kernel.document_immutable();
GRANT INSERT ON kernel.document, kernel.document_line, kernel.document_link, kernel.document_state_history TO
    app_rw;
GRANT UPDATE (status) ON kernel.document TO app_rw;      -- lines: INSERT only, never UPDATE

// kernel.internal.IssuanceProtocol.issue(Document draft, ScopeContext ctx)  -- one transaction (doc 18 5.5)
// 1. validator chain: type-specific validator registered by the owning module (parties, lines, ceilings)
// 2. series = NumberingService.seriesFor(draft.typeCode, ctx.entityId, draft.locationId,
//    draft.tillPositionId)
//    number = UPDATE kernel.numbering_series SET next_number = next_number + 1
//    WHERE series_id = :id RETURNING next_number - 1      -- row lock = gapless, monotonic
//    (offline scopes are never issued here; they arrive via ingestion with the number already assigned)
// 3. freeze totals; content_hash = Envelope.canonicalHash(header + lines); issued_at = clock.now()
//    insert document + lines; insert state row DRAFT->ISSUED; audit.record(DOCUMENT_ISSUED ...)
// 4. owning module's post-issue hook (stock movements etc.) in the same transaction
// 5. events.publish(document.issued.v1 + type-specific event)
// NumberingService: registerSeries(scope, typeCode) on entity/location/position events from M1;
//    holderChange(series, device) audited; closeSeries on position retirement.
// GapCheckJob (nightly): per series, verify density of doc_number and of (source, source_seq); REVIEW on gap
//    unless explained by sync_quarantine (doc 18 C-I2, E-I2).
```

Aspect	Design
Document links	DocumentLinks.link(from, to, type, amount?) validates the rule per type: REVERSES at most once per original and only while reversible per the type's state machine (a predicate the owning module registers); SETTLES requires an amount not exceeding the open balance.
Type registry	document_type seed rows for ORD, DN, GRN, INV, CN, DN2, PRC, RCT, WOF, CNT, ADJ, RPK, XFR, OPB with series scope, issuer role, bilateral, fiscal and offline flags (doc 18 §5); the fiscal flags are revisited by 24B.

Aspect	Design
Tests	Concurrent issuance on one series from two instances yields dense numbers; a crash between number and insert leaves neither (same transaction); UPDATE of a line fails; REVERSES twice fails; gap check flags a fabricated gap and accepts one explained by quarantine.

8. Edge sync gateway

Restated from doc 32 §3, 5, 6, 8. The REST surface is `openapi/sync.yaml`; the gateway runs in the web profile, ingestion in worker or ingest.

Endpoint	Handler	Notes
POST /sync/v1/devices/{id}/enrol	EnrolmentService	Creates the cursor row, registers series through NumberingService, returns starting seq and snapshot pointer
POST /sync/v1/devices/{id}/batches	BatchIngestor	Section 8.1; 409 on sequence mismatch or concurrent batch; 429 from RateLimiter
GET /sync/v1/locations/{id}/snapshot?since=N	SnapshotBuilder	Delta or full-snapshot pointer; Section 8.2
POST /sync/v1/devices/{id}/heartbeat	HeartbeatService	Updates device last_seen, app_version, freshness; returns clock offset, current snapshot version, pending instructions
POST /sync/v1/attachments/presign	PresignService	Section 9

```
// BatchIngestor.ingest(deviceId, batch)                                -- doc 32 3.3, one call per batch
// auth: device token ACTIVE; app_version >= floor else 426; sizes within DR-1 limits
// tx0: cursor = SELECT * FROM kernel.device_sync_cursor WHERE device_id=? FOR UPDATE NOWAIT (409 if locked)
//     if batch.firstSeq > cursor.lastApplied+1 -> 409 {expectedSeq}
//     if batch.batchId == cursor.lastBatchId -> return stored ack (idempotent replay)
// for chunk in batch.events (50 per tx, in order):
//     tx: for e in chunk:
//         if e.seq <= cursor.lastApplied: outcome DUPLICATE; continue
//         if inbox has e.eventId at different seq: quarantine(DUPLICATE_ID); outcome QUARANTINED; continue
//         if !schema.valid(e) : quarantine(SCHEMA); continue
//         if bundle && hash(e) != e.contentHash: quarantine(HASH); continue
//         handler = registry.forType(e.type) // kernel document base for bundles, module handlers for the
//     rest
//         flags = handler.apply(e, systemScope(device)) // never throws for business reasons; returns flags
//         if flags: audit ALERT/REVIEW per flag; line flags stored by the handler
//         outbox.writeFromDevice(e) // re-publish into the backbone with source = device, seq =
//     e.seq
//     inbox.record("sync", e.eventId)
//     cursor.lastApplied = e.seq
//     UPDATE cursor SET last_applied_seq, last_batch_id ...; commit
//     ack = {batchId, lastAppliedSeq, outcomes[], serverTime, clockOffsetMs, snapshotVersion, instructions[]}
//     store ack summary on the cursor for replay answers
```

Aspect	Design
Chunk transaction	50 events per transaction (doc 32 DR-5) so a crash costs at most one chunk; the cursor advances only on commit.
Bundle handler	DocumentBundleHandler inserts header, lines, links, state row and attachment references through the document base (number already assigned by the till; origin = OFFLINE), then calls the owning module's post-ingest hook (stock movements, session facts) and re-validation with the shared engine at the reported snapshot_version_in_use and engine_version (ADR-31).
Snapshot builder	SnapshotBuilder.delta(location, since) reads snapshot_change_log rows since+1..current grouped by table into upserts and tombstones with the payload fetched from the owning module's snapshot query (each of M1, M2, M3, M5, M7 and the kernel registers a SnapshotContributor); if since is older than the change-log window, returns the pointer to the latest full snapshot file; SnapshotRebuildJob writes full snapshots per location nightly and on demand to object storage with a manifest.
Change log producers	Every contributor module publishes a change-log row inside its own transaction through SnapshotChangeLog.record(location(s), table, rowId, op, applyFrom, urgent); the kernel fans one central change (a shared SKU) out to all affected locations via a worker consumer of the module's event, so hot paths do not write thousands of rows synchronously.
Rate limiter	Token buckets per device (batches per minute, bytes per hour) in memory per instance plus a global concurrency limit; fairness by round-robin across devices in the ingest queue.
Tests	The thirteen-case conformance suite of doc 32 §11 against a till simulator in backend/app/src/test/.../sync/TillSimulator.java; two-instance ingestion of one device alternating instances.

9. Attachments

Aspect	Design
Presign	<code>PresignService.put(documentId, attachmentId, contentType)</code> returns a pre-signed PUT URL (15 minutes) for key <code>attachments/{entity}/{documentId}/{attachmentId}</code> ; the S3 client is provider-neutral (path-style addressing on MinIO).
Verification	<code>AttachmentVerifier</code> (job, every 5 minutes): for PENDING rows, HEAD the object, compare hash, set COMPLETE or FAILED; <code>attachment.completed.v1</code> ; M5's write-off approval reads the status (<i>doc 32 §4</i>).
Reads	Pre-signed GET with scope check through the document's RLS: a user who cannot read the document cannot obtain the URL.

10. Notifications delivery

Restated from doc 19 §7. Rules and templates are M9's data; the kernel is the consumer that evaluates and sends.

```
CREATE TABLE kernel.notification_log (  
  notification_id uuid PRIMARY KEY, rule_id uuid NOT NULL, event_id uuid NOT NULL,  
  recipient_hash char(64) NOT NULL, channel text NOT NULL, rendered_length int,  
  status text NOT NULL, provider_ref text, attempts smallint NOT NULL DEFAULT 0,  
  created_at timestamptz NOT NULL DEFAULT now(), last_attempt_at timestamptz,  
  UNIQUE (rule_id, event_id, recipient_hash));
```

Aspect	Design
Dispatcher	<code>@EventConsumer(types = "*", consumer = "kernel.notifications")</code> matches each event against active rules (cached, invalidated by <code>notification_rule.changed.v1</code>); resolves the audience through M1 queries; renders with <code>IcuMessages</code> in the recipient's language; inserts the log row (the unique constraint is the idempotency); hands to <code>NotificationChannel</code> (M9's adapter interface: SMS, EMAIL, IN_APP).
Retry and suppression	RetrySweep job every 2 minutes: FAILED with attempts under 3 and age under 24 h retried with backoff; secondary provider after the third; quiet hours and hourly de-duplication checked before send; entity kill switch from the configuration register.
Tests	Replayed event sends once; quiet-hours suppression; fallback language; provider failover.

11. Configuration register

Restated from doc 19 §8.

```
CREATE TABLE kernel.config_item (key text PRIMARY KEY, value_type text NOT NULL, schema jsonb NOT NULL,
description_en text NOT NULL, description_si text, description_ta text, default_value jsonb NOT NULL,
scope_kind text NOT NULL CHECK (scope_kind IN ('FEDERATION','ENTITY','LOCATION')),
change_permission text NOT NULL, sensitive boolean NOT NULL DEFAULT false, till_visible boolean NOT NULL
DEFAULT false);
CREATE TABLE kernel.config_value (key text NOT NULL REFERENCES kernel.config_item, scope_entity_id uuid,
scope_location_id uuid, value jsonb NOT NULL, effective_from timestampz NOT NULL DEFAULT now(),
changed_by uuid NOT NULL, changed_at timestampz NOT NULL DEFAULT now(),
PRIMARY KEY (key, coalesce(scope_entity_id,'00000000-0000-0000-0000-000000000000')::uuid),
coalesce(scope_location_id,'00000000-0000-0000-0000-000000000000')::uuid), effective_from));
```

Aspect	Design
Resolver	ConfigResolver.get(key, entityId, locationId): latest effective row at location, else entity, else default; typed accessors getInt / getDuration / getBool / getJson validated against the item schema; per-instance cache invalidated by config.changed.v1.
Setter	ConfigRegistry.set(key, scope, value, ctx, reason) checks the item's change_permission (MFA if sensitive), validates against schema, inserts a new row (append-only), audits with before/after, publishes the event and, for till_visible keys, a change-log row for the affected locations.
Seed	seed/config-items.yaml with the doc 07 §4 register plus doc 19 §8 and doc 21 §7 items; SeedLoader upserts items on start (never values).
Tests	Resolution precedence; schema rejection; till_visible fan-out to the change log; MFA on sensitive keys.

12. Scheduling

Restated from doc 19 §9, doc 14 §1.3.

```
CREATE TABLE kernel.scheduled_job (name text PRIMARY KEY, module text NOT NULL, cron text, continuous boolean
NOT NULL DEFAULT false,
lock_timeout interval NOT NULL, max_runtime interval NOT NULL, critical boolean NOT NULL DEFAULT false,
enabled boolean NOT NULL DEFAULT true);
CREATE TABLE kernel.job_run (run_id uuid PRIMARY KEY, name text NOT NULL REFERENCES kernel.scheduled_job,
started_at timestampz NOT NULL,
finished_at timestampz, instance text NOT NULL, outcome text, items_processed int, error text);
CREATE TABLE kernel.shedlock (name text PRIMARY KEY, lock_until timestampz NOT NULL, locked_at timestampz
NOT NULL, locked_by text NOT NULL);
```

Aspect	Design
Registration	@ScheduledJob(name, cron continuous, lockTimeout, maxRuntime, critical) on a bean method; JobRegistry upserts scheduled_job at start (never flips enabled); the fleet console flips it.
Runner	JobRunner uses ShedLock's JDBC provider with lockAtMostFor = lockTimeout; a second instance skips; a job over maxRuntime is interrupted and its run marked TIMEOUT with an ALERT; critical jobs retry immediately once.
Day-close trigger	DayCloseTrigger consumes till_session.closed.v1; when every open session at a location has closed (or at the location's configured cut-off), it advances the location's business date and publishes location.day_closed.v1, which end-of-day aggregation and apply_from activation consume (doc 32 §5.2).
Phase-1 job seed	outbox-relay (continuous), partition-maintenance (daily), gap-check (nightly), attachment-verify (5 min), notification-retry (2 min), snapshot-rebuild (nightly + on demand), changelog-prune (nightly), idempotency-expiry (hourly), end-of-day (per day_closed), expiry-scan (nightly), stock-reconcile (nightly), ring-promotion (hourly).
Tests	Two instances, one run per schedule; timeout handling; manual run permission and audit; day-close advances exactly once per location per day.

13. Business date and clock

Aspect	Design
Clock	An injectable Clock (UTC) everywhere; tests use a fixed clock. ClockOffset keeps per-device offsets from heartbeats and corrects occurred_local into occurred_at on ingestion when the till marks the event as offline-timed.
Business date	BusinessDateService.current(locationId) from the location's day-close state; document.business_date is set from it at issuance (central) or taken from the till's value (ingestion), never from the wall clock.

14. Tickets

#	Ticket	Est. (days)	Done when
K-01	Scope filter, connection customizer, granted entities, PARTY policy template and masking view convention	1.5	RLS matrix incl. PARTY and NONE passes
K-02	JWT claims mapping, provider mapper endpoint, Keycloak admin wrapper with scope rule, step-up interceptor, device auth, PIN hasher	3	Hello module logs in as each principal; step-up round trip in the web shell
K-03	Permission resolver and cache, command interceptor order, SoD helpers, till snapshot signer	2	Interceptor tests; snapshot verifiable on the till
K-04	Audit tables, facade, correlation, exception queue query, partition job, privilege test	1.5	One row per command; UPDATE denied
K-05	Outbox writer, relay, broker adapter, consumer framework with inbox, dead-letter, replayer	3	Two-instance exactly-once test; DLQ; replay
K-06	ICU messages with fallback, NFC converter, collation and search helpers, formats, A4 renderer worker	2.5	Golden trilingual PDF; sort test set
K-07	Document tables and DDL, immutability trigger, issuance protocol, numbering service, links, gap-check job, type seed	3	Concurrent issuance dense; immutability; links rules
K-08	Sync API, enrolment, batch ingestor with quarantine, snapshot builder and contributors, change-log fan-out, heartbeat, rate limiter, till simulator	5	Doc 32 conformance suite green
K-09	Attachment presign and verifier	0.5	PENDING→COMPLETE on upload
K-10	Notification dispatcher, log, retry sweep, channel interface	1.5	Once per event; suppression
K-11	Config item and value tables, resolver, cache, setter with audit, seed loader	1.5	Precedence and schema tests
K-12	Scheduled job tables, runner with ShedLock, registry, day-close trigger, phase-1 job seed	1.5	One run across two instances
K-13	Clock, offset correction, business date service	0.5	Business date from day-close

Total about 27 developer-days. With 17A's 18.5 that is about 45 for the platform pair: Sprint 0 runs three weeks, not two (8–25 September), or a third developer joins for K-05, K-07 and K-08. Doc 20's schedule already carries a reserve; this is the first measured correction and should be recorded there.

15. Open points taken as assumptions

Point	Assumed	Source
Broker	RabbitMQ quorum queues behind BrokerAdapter	Doc 19 DR-2
Identity provider	Keycloak behind IdentityProviderClient; provider mapper reads user_role at token issue	Doc 19 DR-1
A4 rendering engine (DR-3 here)	Headless Chromium worker with bundled Noto fonts; revisit if the operational footprint is judged too heavy for the single-DevOps model	Platform pair, DevOps
Snapshot fan-out	Worker-side fan-out of central changes to per-location change-log rows; hot paths never write per-location rows synchronously	Doc 32 §5.2
FY reset and GRN fiscal scope	No reset; GRN non-fiscal; document_type seed flags accordingly	Tax advisor (24B)
PARTY masking views	Convention defined here; M4 implements against its document extension tables	Doc 24

End of document. 19A v0.1; once Sprint 0 completes, the kernel package's own README and tests are the living version of this guide.